Spring 2026 - Data Engineering: Project Ideas

The goal of your final project is to give you hands-on experience in developing data engineering approaches and applying data engineering methods.

You can propose your own project, which should:

- 2 or more data sources
- Use data engineering methods you have learned (or will learn) in class.

For example, you can propose a task/scientific questions for which you will construct a data product that combines multiple datasets that you will integrate and clean.

You can also choose one of the projects described below. Note that these are project ideas, and there is room for creativity and to further specify what you will do.

Project Mechanics:

- You will form teams of at least 2 and at most 3 people
- The deliverables include a project report (which you may turn into a paper) and all artifacts (code, data) required to reproduce your results.
- You will also present your project and findings to the class.

Project 1: LLM-Powered Data Triage

Motivation: Understanding Wildlife Trafficking. Wildlife trafficking remains a critical global issue, significantly impacting biodiversity, ecological stability, and public health. Despite efforts to combat this illicit trade, the rise of e-commerce platforms has made it easier to sell wildlife products, putting new pressure on wild populations of endangered and threatened species. The use of these platforms also opens a new opportunity: as criminals sell wildlife products online, they leave digital traces of their activity that can provide insights into trafficking activities as well as how they can be disrupted. The challenge lies in finding these traces. Online marketplaces publish ads for a plethora of products, and identifying ads for wildlife-related products is like finding a needle in a haystack. Learning classifiers can automate ad identification, but creating them requires costly, time-consuming data labeling that hinders support for diverse ads and research questions.

Approach: LLM-Powered Data Triage. While large language models (LLMs) can directly label advertisements, doing so at scale is prohibitively expensive. Lean-to-Sample (LTS) is a new approach that automatically generates learning classifiers for triage tasks like finding ads on eBay that contain animals or animal related products. LTS combines clustering with a multi-armed bandit algorithm to obtain diverse, representative samples; and leverages LLMs to label these samples, using active-learning to reduce training costs.

Project Goals: LTS was evaluated using eBay ads and shown to derive effective classifiers for different tasks related to identifying wildlife-related products. In this project, you will explore the

effectiveness of LTS for other data and triage tasks on datasets with class imbalance. You will conduct a detailed analysis of the suitability (and limitations) of the approach and LLMs for these tasks, and, based on your findings, you will extend or customize the system for these tasks. The goal is to have a general framework that supports a wide range of data triage tasks.

Notes: You can use standard classification benchmarks, including benchmarks that include class imbalance, such as:

- Reuters-21578: A collection of newswire articles commonly used for text classification. Subsets like R8 and R52, which have different degrees of class imbalance. https://www.kaggle.com/datasets/thedevastator/uncovering-financial-insights-with-the-reuters-2
- WebKB: Composed of web pages from university computer science departments, this dataset contains inherent class imbalance. It has four main categories: student, faculty, course, and project. https://www.cs.cmu.edu/afs/cs.cmu.edu/project/theo-20/www/data/
- Emotions: A dataset of tweets classified into different emotion categories, often exhibiting a severe class imbalance. For example, the "Joy" category is typically much more frequent than "Disgust". https://www.kaggle.com/datasets/praveengovi/emotions-dataset-for-nlp
- 20 newsgroups. https://www.kaggle.com/datasets/crawford/20-newsgroups

In addition to using the test data provided in these benchmarks, you will use an LTS to create a sample of the data and an LLM to label the sample.

You are encouraged to find additional datasets and related work.

References:

- A Cost-Effective LLM-based Approach to Identify Wildlife Trafficking in Online Marketplaces. https://dl.acm.org/doi/10.1145/3725256
- https://github.com/VIDA-NYU/LTS

Project 2: Dataset Description Generation

Motivation: We have witnessed a proliferation of data portals and data lakes as more structured data is generated and made available. It is estimated that there are tens of millions of datasets on the web. While these datasets present significant opportunities for data-driven insights, they are difficult to find. Many datasets are published with inadequate or missing descriptions. In the Auctus dataset search engine index, 3,121 out of 23,520 datasets (13.2%) have no descriptions, and 2,346 datasets (approximately 10%) have descriptions with 10 words or fewer. Since dataset search interfaces rely on the descriptions to answer queries, datasets without an appropriate description are effectively invisible.

Approach: LLM-Powered Dataset Description Generation. AutoDDG is an end-to-end framework for the automatic generation of tabular dataset descriptions. Given an input table, AutoDDG profiles the data to extract two complementary summaries: a content and a semantic

summary. The content summary is generated using algorithmic data profiling techniques, capturing essential dataset characteristics such as attribute types, statistical summaries, and distributions. The semantic summary is derived with the assistance of an LLM and includes contextual information that is not explicitly present in the data, such as the topic of the dataset and types of analyses the dataset can be used for. These summaries are input to an LLM which is instructed to generate a readable and concise yet comprehensive textual summary of the dataset.

Project Ideas: In this project, you can explore different directions:

1. The application and customization of this approach to specific repositories and domains. For example, to generate descriptions for datasets in NYC Open Data, Zenodo. Besides customizing the descriptions and evaluating their quality, you will also explore the scalability of the approach to large dataset collections.
2. Extend the approach to leverage other information sources, in addition to the contents and LLMs, to enrich the descriptions, such as for datasets published with scientific articles, use the article contents. The datasets in https://opportunityinsights.org/data were derived for analyses that are described in a series of scientific articles; some of the datasets also include descriptions as well as stata scripts. You could experiment with the automatic generation of descriptions for these datasets and compare them with the original descriptions.
3. Generate descriptions for multimodal datasets encompassing tabular, imaging, and text data. Multi-modality refers to the ability of artificial intelligence systems to understand, interpret, and generate outputs across multiple types of data inputs, such as text, images, audio, and video.

[1] AutoDDG: Automated Dataset Description Generation using Large Language Models https://arxiv.org/abs/2502.01050
[2] Radford, Alec, et al. "Learning transferable visual models from natural language supervision." International conference on machine learning. PMLR, 2021.
[3] Eggert, Gus, et al. "TabLib: A Dataset of 627M Tables with Context." arXiv preprint arXiv:2310.07875 (2023).

Project 3: Spatial Data Discovery: Spatial-Correlation Joins

Motivation and Background: The ability to discover correlated data is useful in many applications, from augmenting training data to improve machine learning models to identifying potential causal relationships.

Correlation joins [1,2] were proposed to support the discovery of correlated data: given a query dataset Q, with an associated key k and a target variable v, and a data collection C, the goal is to find datasets D in C such that Q and D can be joined on k and D contains an attribute that is correlated with *v*. Correlation Sketches were proposed to efficiently evaluate correlation join queries, by simultaneously summarizing information about joinability and correlation, allowing the estimation of different correlations measures between columns of unjoined datasets.

Project Goals: You will develop a new type of correlation sketches that supports join correlation queries over spatial data, i.e., where the key is a spatial attribute. Specifically, you will explore the use of geohashes [4] and the ability to support queries that span multiple spatial granularities, e.g., given a query dataset at the neighborhood level, find correlated datasets at the lat-long level (or vice versa).

You will compare the performance of your approach against Nexus [3], , a system designed to align large repositories of spatio-temporal datasets and identify correlations.

References:

[1] *Aécio Santos*, Aline Bessa, Christopher Musco, Juliana Freire:
A Sketch-based Index for Correlated Dataset Search. ICDE 2022: 2928-2941

[2] Aécio Santos, Aline Bessa, Fernando Chirigati, Christopher Musco, Juliana Freire:
Correlation Sketches for Approximate Join-Correlation Queries. SIGMOD Conference 2021: 1531-1544

[3] Nexus: Correlation Discovery over Collections of Spatio-Temporal Tabular Data. https://raulcastrofernandez.com/papers/nexus-sigmod24.pdf

[4] https://en.wikipedia.org/wiki/Geohash

Project 4: Semantic Joins and Value Matching Operators

Semantic joins extend traditional equi-joins by considering discrepancies between matching values. For example, consider the following two columns storing country names:

An equi-join operation would miss the value matches between USA - United States of America and Netherlands - The Netherlands, due to their discrete formats. Therefore, a semantic join operator requires finding all valid value matches, beyond only exact ones, between the value sets of the columns.

Challenges: Finding value matches to perform a semantic join operation is a widely-studied problem, but remains a challenging task for the following reasons:

i) One way to deal with the problem is to define a set of transformation functions and use a combination of those to transform a value to the same format as the other one [1, 2] , e.g., 'The Netherlands' can become 'Netherlands' by dropping 'The '. Such techniques have been shown to be highly precise, but come with low recall, as they cover only a portion of possible value matches (e.g., in our example USA and United States of America cannot be joined using syntactic transformation functions).

ii) Another way is to define one or more similarity functions between values and use a threshold to accept matches - this transforms the problem to *similarity joins* [3]. For example, using *edit distance*, 'The Netherlands' is very close to 'Netherlands' and possibly qualifies as a valid value match pair. Nonetheless, this might not work for value matches such as 'USA' - 'United States of America' due to high edit distance (false negatives), or even point to false positives ('United Kingdom' - 'United States of America' because they share 'United'). Furthermore, setting the right threshold depends on the underlying data, making such a solution not practical in realistic scenarios.

iii) Recently, works that leverage LLMs attempt to build methods that transform values to a target format [4]. Such methods could be used when columns use a uniform representation for their values (e.g., ISO-codes for country names), since the LLM can be used in a few-shot fashion to transform values from one column to another. While this direction seems very promising, it assumes uniformity in value representation for at least one of the columns and that some value match examples exist.

iv) Several value matches might require external knowledge or nuanced semantics in order to be captured, e.g. 'USA' - 'United States of America', 'South Korea'- 'Republic of Korea' etc. LLMs could enable us to capture such cases, yet we cannot only rely on using LLMs to perform semantic joins, since the large number of possible value pairs between columns can lead to high computational and monetary costs (if we assume 1 LLM prompt / value pair). A recent work [5] deals with this issue and investigates the efficient use of LLMs to perform semantic joins on any given predicate expressed in natural language - in our case the predicate is semantic equivalence of values.

Goal: Build a generic semantic join operator which attempts to maximize the number of valid value matches captured between the values sets of two columns (i.e., a method that has both high precision and recall). The operator should be able to capture various types of value discrepancy - both syntactic (e.g., typos, abbreviations, and other syntactic-based transformations) and semantic (e.g., country names to iso country codes). Towards this direction, you are encouraged to combine existing solutions and/or extend them. Notably, the method should also be efficient and account for joins between columns with potentially thousands of unique values.

Resources: The benchmark from [2] can be used for evaluation, as it contains column pairs that can be joined using a semantic join operator and ground truth of value-matches between them. The ground truth of joins from the Freyja benchmark can be also used to measure

effectiveness/efficiency, yet these don't come with ground truth of value matches - in this case the students will have to perform manual evaluation. Method implementations can be found in the referenced works.

Outcomes: A code repository with the suggested method implementation and clear instructions on how to use it, together with a report explaining the workings of the method and discussing evaluation results, insights and potential limitations. Your method could be potentially integrated into our open-source data integration framework BDIKit.

Remarks: Given the challenging nature of building a generic, effective and efficient semantic join operator, students will be evaluated primarily on their understanding of the problem, the soundness of their proposed approach, the validity of their methodology, and the insights gained from their experiments.

References

[1] Zhu, Erkang, Yeye He, and Surajit Chaudhuri. "**Auto-join: Joining tables by leveraging transformations.**" *Proceedings of the VLDB Endowment* 10.10 (2017): 1034-1045.
[2] Li, Peng, et al. "**Auto-fuzzyjoin: Auto-program fuzzy similarity joins without labeled examples.**" *Proceedings of the 2021 international conference on management of data*. 2021.
[3] Xu, Pengfei, and Jiaheng Lu. "**Towards a unified framework for string similarity joins.**" *Proceedings of the VLDB Endowment* 12.11 (2019): 1289-1302.
[4] Dargahi Nobari, Arash, and Davood Rafiei. "**Dtt: An example-driven tabular transformer for joinability by leveraging large language models.**" *Proceedings of the ACM on Management of Data* 2.1 (2024): 1-24.
[5] Trummer, Immanuel. "**Implementing Semantic Join Operators Efficiently**." *arXiv preprint arXiv:2510.08489* (2025).

Project 5: Open Urban Data: Understanding the Landscape

A growing number of cities are now making urban data freely available to the public. Besides promoting transparency, these data can have a transformative effect in social science research as well as in how citizens participate in governance. In a 2014 study (see below), to better understand the landscape of open urban data, Barbosa et al. examined multiple repositories and reported on different aspects of the data that they store, including the number of data sets, their content, and popularity. They also investigated opportunities for integrating these data, i.e., which datasets could be joined.

The goal of this project is to perform a new study that reflects the current state of open urban data. Since the original study was published, not only has the number of available datasets increased, but there are also new techniques that enable a more detailed analysis. For example, strategies such as Lazo and semantic joins [3], make it possible to obtain a much

better picture of joinability among a large number of datasets; open-source profilers and LLMs are also able to detect semantic types with greater accuracy.

References:

1. Structured Open Urban Data: Understanding the Landscape. Barbosa et al., https://www.liebertpub.com/doi/10.1089/big.2014.0020
2. Lazo: A Cardinality-Based Method for Coupled Estimation of Jaccard Similarity and Containment. https://ieeexplore.ieee.org/document/8731486
3. DeepJoin: Joinable Table Discovery with Pre-trained Language Models. https://www.vldb.org/pvldb/vol16/p2458-dong.pdf

Project 6: Research Replication

You can select a scientific paper that includes data-driven experiments, reproduce it and critically analyze the results, and replicate their experiments using additional data. The goal is to both assess the validity/correctness of the reported results and the generalizability of the proposed approach/analysis/claims.

Here are some examples:

- This paper https://raulcastrofernandez.com/papers/nexus-sigmod24.pdf (https://github.com/TheDataStation/nexus_correlation_discovery) discusses experimental results using Chigago Open Data. To assess its generalizability, you could also run the approach using NYC Open Data.
- There are a number of different data-driven studies in https://opportunityinsights.org (e.g., https://academic.oup.com/qje/article/133/3/1107/4850660?login=true#supplementary-data) and associated press coverage (https://opportunityinsights.org/press)

Project 7: Semantic Type Discovery and Annotation

Understanding dataset semantics is crucial for effective search, discovery, and integration pipelines. To this end, column type annotation (CTA) methods associate columns of tabular datasets with semantic types that accurately describe their contents, using pre-trained deep learning models or Large Language Models (LLMs). However, existing approaches require users to specify a closed set of semantic types either at training or inference time, hindering their application to domain-specific datasets where pre-defined labels often lack adequate coverage and specificity. Furthermore, real-world datasets frequently contain columns with values belonging to multiple semantic types, violating the single-type assumption of existing CTA methods. While proprietary LLMs have shown effectiveness for CTA, they incur high

monetary costs and produce inconsistent outputs for similar columns, leading to type redundancy that negatively affects downstream applications.

StraTyper is a cost-effective method for column type discovery (CTD) and multi-type annotation (CMTA) in dataset collections. It eliminates the need for pre-defined semantic labels by systematically employing LLMs to discover types tailored to the dataset collection at hand. Through strategic column clustering, controlled type generation, and iterative cascading discovery, StraTyper balances type precision with annotation coverage while minimizing LLM costs. Experimental evaluation—both manual and LLM-assisted—on real-world benchmarks demonstrates that StraTyper discovers accurate types for both numerical and non-numerical data, achieves substantial cost savings compared to commercial LLMs, and effectively handles multi-typed columns. StraTyper's annotations improve downstream tasks, including join discovery and schema matching, outperforming LLM-only baselines.

Idea 1: StraTyper was evaluated on two dataset collections. You will select other collections, evaluate the approach for these, and suggest/implement any required improvements. For example, you could select another category from NYC Open Data (e.g., Environment, Health) or from other open data repositories such as https://data.cityofchicago.org

Idea 2: You will use the types inferred by StraTyper (available on github) for the NYCOpenData data collections to identify and rectify data quality issues in the datasets. For example, detecting columns where the inferred type contradicts the stored values, finding columns that appear across datasets but with inconsistent formatting, or flagging columns with high null rates within a given semantic type.

Idea 3: You can inspect the clustering techniques of StraTyper and implement alternatives that potentially bring better results. In addition, the current version of StraTyper uses column name-based clusters in conjunction with value-based ones, deeming it inapplicable when headers are not available or are incomprehensible. Therefore, you could implement a clustering method that substitutes the column name-based clusters.

References:

1. StraTyper: Automated Semantic Type Discovery and Multi-Type Annotation for Dataset Collections. https://arxiv.org/abs/2602.04004
2. https://github.com/VIDA-NYU/stratyper
3. See data cleaning lecture notes and references.

Project 8: Data Discovery in Scientific Publications

Researchers, librarians, and data curators spend significant time manually identifying dataset references embedded in scientific publications — scattered across data availability statements, figure captions, inline text, and supplementary materials. This project asks students to build a dataset reference extraction pipeline using DocETL [1] and rigorously compare it against DataGatherer [2], a purpose-built LLM-powered tool for the same task.

DataGatherer is a specialized system that extracts structured dataset records (identifier + repository) from scientific papers. It supports two strategies:

- *Full-Document Read (FDR)*: feeds the full preprocessed HTML of a paper to an LLM.
- *Retrieve-Then-Read (RTR)*: first uses rule-based CSS/XPath selectors to locate relevant sections (e.g., Data Availability Statements), then extracts from those sections only.

DocETL is a declarative, LLM-powered data processing framework for complex document analysis pipelines. Rather than writing custom extraction code, users compose pipelines from reusable operators (map, filter, reduce, resolve, split, gather) over a collection of documents. DocETL handles chunking long documents, optimizing prompts, and resolving entities across documents.

Goals:

1. Build a DocETL pipeline that takes a corpus of scientific papers (PDFs or HTML) as input and outputs structured dataset references `{dataset_identifier, repository, url}` for each paper.
2. Evaluate the pipeline on the DataRef-EXP and/or DataRef-REV benchmarks provided by the DataGatherer authors.
3. Compare DocETL vs. DataGatherer across multiple dimensions - accuracy, coverage, cost, engineering effort, discussion on failure cases for both methods etc.
4. Reflect critically on the tradeoffs between general-purpose declarative pipelines and specialized tools.

Resources:

- DataGatherer code: https://github.com/VIDA-NYU/data-gatherer
- Benchmark datasets (DataRef-EXP, DataRef-REV): https://doi.org/10.5281/zenodo.15549086
- DocETL documentation: https://ucbepic.github.io/docetl/

References:

[1] Shankar, Shreya, et al. "DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing." Proceedings of the VLDB Endowment 18.9 (2025): 3035-3048.

[2] Marini, Pietro, et al. "Data gatherer: LLM-powered dataset reference extraction from scientific literature." Proceedings of the Fifth Workshop on Scholarly Document Processing (SDP 2025). 2025.